

D598 Task 3: Presentation

A. Explain how the code works for the program you submitted in Task 2.

At the start of the program for Task 2, the necessary libraries were imported to support the data operations required to accomplish the task.

The Pandas library was imported as pd. It is beneficial for working with data frames, which is a requirement for Task 2.

I also imported the display function from IPython.display. This helps in Jupyter Notebook by allowing dataframes to be rendered in a more readable and formatted way compared to the standard print statements.

```
#Cody Pace  
#WGU D598 Task 2  
#August 2025  
  
#Import Libraries  
import pandas as pd  
from IPython.display import display
```

Next, I imported the provided Excel file, "D598 Data Set.xlsx", using pd.read_excel() from the Pandas library. Using this function, the code reads in the contents of the Excel file and loads it into a dataframe I named df_input. Since the file is stored in the same directory as the Python script, I did not need to provide the full file path.

```
#Import the Excel file as a data frame  
df_input = pd.read_excel("D598 Data Set.xlsx")
```

Next, I needed to identify duplicate records within the `df_input` dataframe. To accomplish this, the `duplicated()` function provided by Pandas was used. These duplicate rows are filtered and stored in a new dataframe called `df_duplicates`.

I created a variable named `duplicates_count` using the `len()` function to count the number of duplicate records found. This count is then printed to provide instant feedback on the presence of duplicates.

To expand on Task 2's requirements, I implemented a check to see if duplicates exist. If duplicates exist, the code prints the Business ID of the duplicate records. This addition would be helpful in a real-world context, where data analysts or upstream providers might need to trace the source of the data issue. The code then removes duplicates from `df_input` using the `drop_duplicates()` method and prints a message indicating how many duplicate records were removed. If no duplicates are found, the program informs the user accordingly.

```
#Identify duplicate records, provide count, Business ID for later triage, then remove the duplicates
df_duplicates = df_input[df_input.duplicated()]
duplicates_count = len(df_duplicates)
print(f'Number of duplicate records: {duplicates_count}')

#Provide the Business ID for the duplicate record for easy review.
if len(df_duplicates) > 0:
    print(df_duplicates[['Business ID']])
    df_input = df_input.drop_duplicates()
    print(f'{duplicates_count} duplicate records removed\n')
else:
    print('No duplicates were found in the file.\n')
```

Here, I grouped the data by the Business State column using Pandas's `groupby()` function. The grouped data is stored in a new dataframe named `df_state_stats`.

In preparation for performing aggregation functions, I also created a list named `df_state_cols`. This list contains the names of the numeric columns I want to analyze. By creating this list, I ensure that the aggregation functions are only applied to relevant numeric columns, not fields such as Business ID.

```
#Group all rows by Business State  
df_state_cols = [ 'Total Long-term Debt', 'Total Equity', 'Debt to Equity',  
                  'Total Liabilities', 'Total Revenue', 'Profit Margin']  
df_state_stats = df_input.groupby('Business State')
```

I performed descriptive statistical operations in this section on the numeric columns defined earlier in `df_state_cols`. The aggregate functions are specified in the list `desc_stats` and applied using the `.agg()` method on the grouped dataframe `df_state_stats`. These results are stored in a new dataframe called `df_desc_stats`. I use a list comprehension to flatten the column headers for improved readability in the Jupyter Notebook. A print statement labels the output, and the `display()` function renders the dataframe in a readable format.

```
#Descriptive statistics (mean, median, min, and max)  
#for Total Long-term Debt, Total Equity, Debt to Equity,  
#Total Liabilities, Total Revenue, Profit Margin  
desc_stats = ['mean', 'median', 'min', 'max']  
df_desc_stats = df_state_stats[df_state_cols].agg(desc_stats)  
df_desc_stats.columns = ['_'.join([col, stat]) for col, stat in df_desc_stats.columns]  
print('Descriptive Statistics by State:')  
display(df_desc_stats)
```

In this code block, I filtered the `df_desc_stats` dataframe to identify the businesses with negative debt-to-equity ratios. In the first line, we see the creation of the dataframe `df_input`. This dataframe is created using the `df_input` dataframe column 'Debt to Equity'. I identified records where the debt-to-equity ratio was less than zero and sorted the values in ascending order by default. A header line is printed for easy location in the Jupyter output, and the indexing is reset to improve the visual and readability. For the output of this dataframe, I found the columns business ID, Business State, and Debt to Equity to be the most important. I then print a summary line to indicate the total count of businesses with a negative debt-to-equity ratio.

```
#Filter df_desc_stats to identify all businesses with debt-to-equity (dbteq) ratios that are negative
df_negative_dbteq = df_input[df_input['Debt to Equity'] < 0].sort_values(by = 'Debt to Equity')
print('Businesses With A Negative Debt To Equity Ratio')
df_negative_dbteq.reset_index(drop=True, inplace=True)
display(df_negative_dbteq[['Business ID', 'Business State', 'Debt to Equity']])
print(f'There are {len(df_negative_dbteq)} businesses with a negative Debt to Equity Ratio\n')
```

The code must divide the long-term debt by revenue to calculate the debt-to-income ratio. We cannot divide by zero; this will cause the code to fail. To handle these troubled accounts and prevent breaking my code, I created a new dataframe named `df_zero_revenue`. This dataframe stores the record in `df_input` where Total Revenue equals zero. Then, if `df_zero_revenue` is not empty, the code will print a warning that businesses in the dataframe have total revenue equal to zero dollars and inform the user that they will be skipped in processing. The business ID is provided for the records that are skipped. This will provide information that can be used later in triage to check the upstream dataset.

```
#Find the Debt-to-income ratio for every business and store in a new data frame.
#Debt-to-income ratio = Long-term debt / revenue

#Find businesses with a Total Revenue = $0
df_zero_revenue = df_input[df_input['Total Revenue'] == 0]

#Warn the user if records are found with $0 Total Revenue. Provide Business ID for triage.
if not df_zero_revenue.empty:
    print('WARNING: The following businesses have a Total Revenue = $0 and will be skipped:')
    display(df_zero_revenue[['Business ID']])
```

Since records with zero total revenue cannot be used to calculate the debt-to-income ratio, I removed them and created a new dataset, `df_filtered_rev`. Then update the `df_input` dataframe to remove these records as well.

```
#Filter records out
df_filtered_rev = df_input[df_input['Total Revenue'] != 0]
df_input = df_filtered_rev.copy()
```

With the dataframe cleaned, the code calculates the debt-to-income ratio. First, it creates a new data frame, `df_debt_income_ratio`, by copying the business ID, total long-term debt, and total revenue from `df_filtered_rev`. Then it uses the columns' total revenue and long-term debt to calculate the debt-to-income ratio as a new field in the `df_debt_income_ratio` dataframe. Sort the `df_debt_income_ratio` by the new field in descending order, print the header label, and reset the index to clean up the display. Finally, provide a display of the new dataframe.

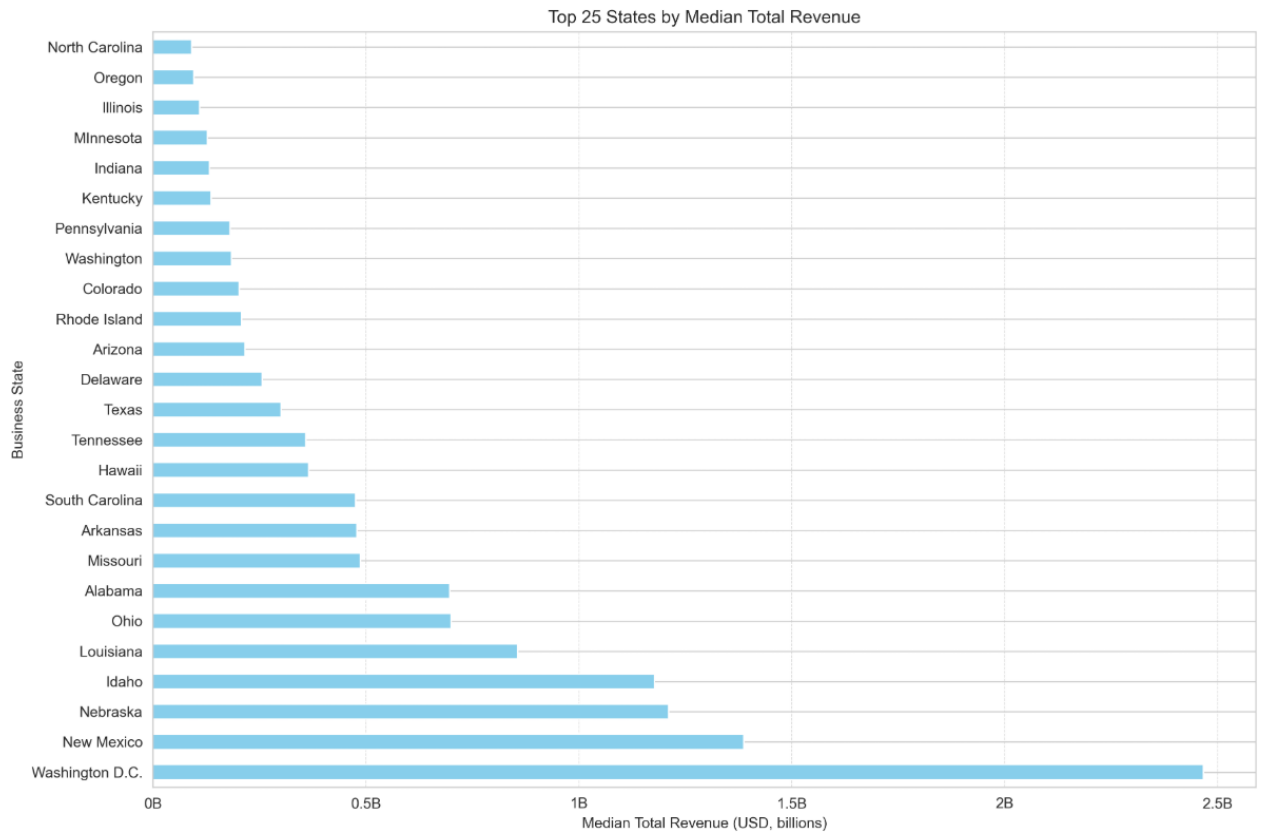
```
#Calculate Debt-to-Income Ratio
df_debt_income_ratio = df_filtered_rev[['Business ID', 'Total Long-term Debt', 'Total Revenue']].copy()
df_debt_income_ratio['Debt-to-Income Ratio'] = df_debt_income_ratio['Total Long-term Debt'] / df_debt_income_ratio['Total Revenue']
df_debt_income_ratio = df_debt_income_ratio.sort_values(by = 'Debt-to-Income Ratio', ascending=False)
print('Debt-to-income ratio for each business')
df_debt_income_ratio.reset_index(drop=True, inplace=True)
display(df_debt_income_ratio)
```

In the final step, `df_final` is created by merging `df_input` with `df_debt_income_ratio`. This merge is performed on the 'Business ID' column using a left join. This ensures all original records are retained while adding the new Debt-to-Income Ratio column to the dataframe. A header label is printed, the index is reset for a cleaner display, and the results are displayed.

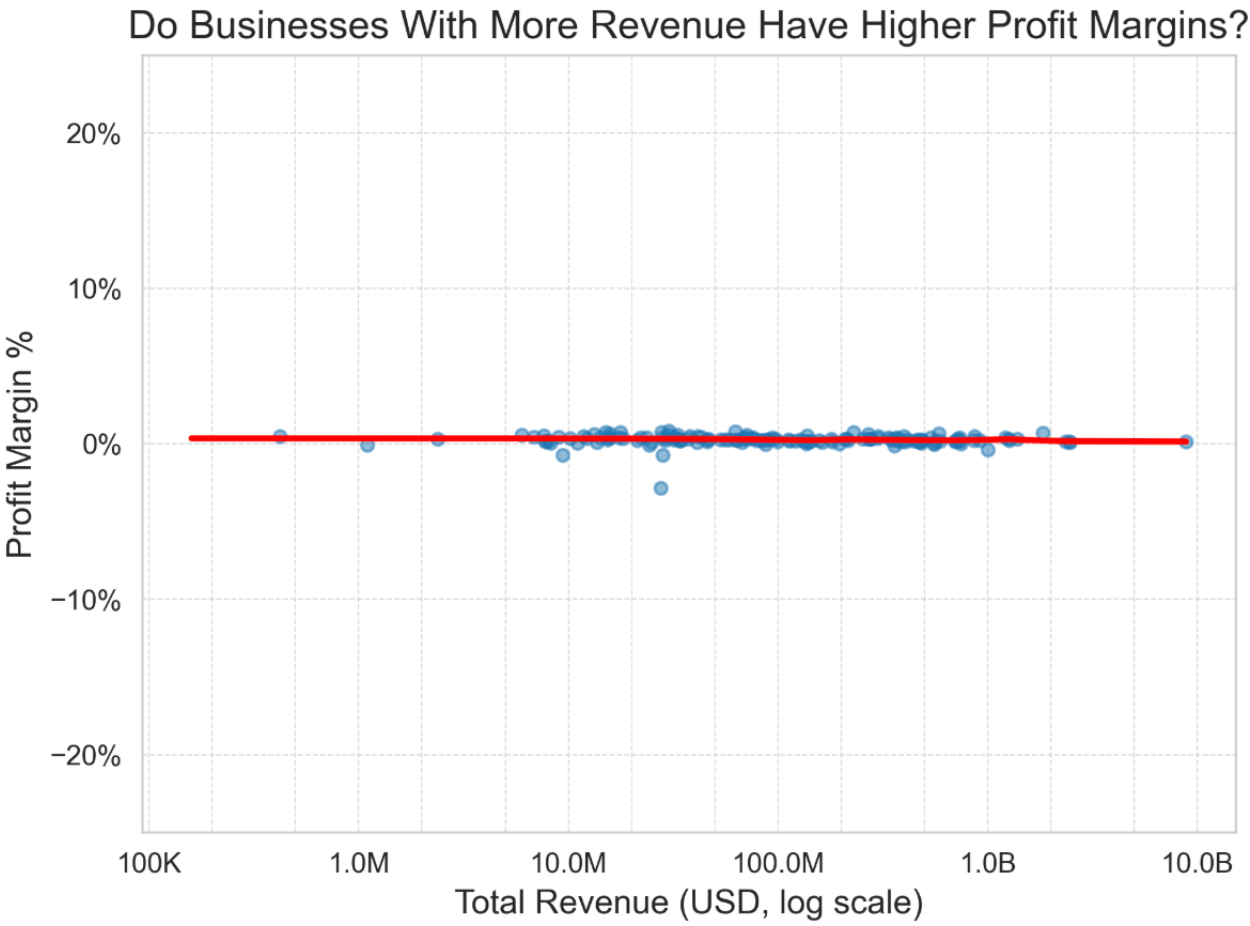
```
#Concatenate the debt-to-income ratio data frame with the original data frame
df_final = pd.merge(df_input, df_debt_income_ratio[['Business ID', 'Debt-to-Income Ratio']], on='Business ID', how='left')
print('Final Output')
df_final.reset_index(drop=True, inplace=True)
display(df_final)
```

B. Provide 4 customized data visualizations.

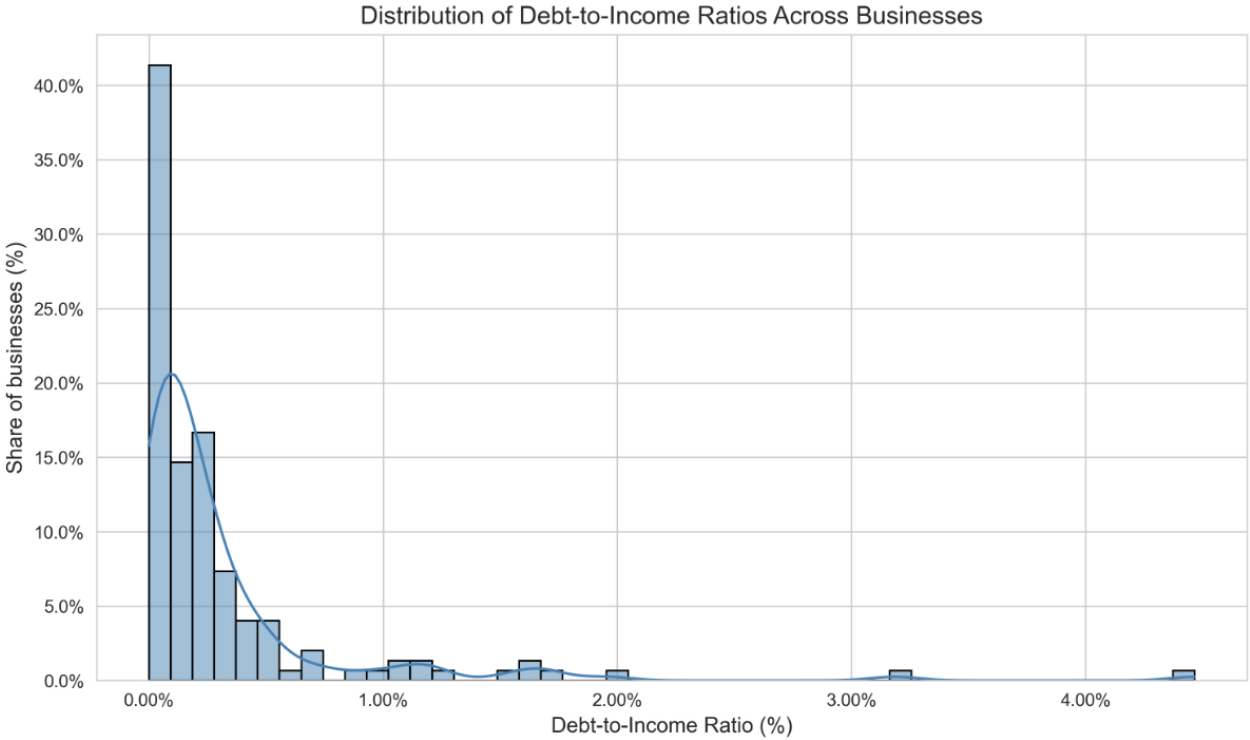
Visualization 1



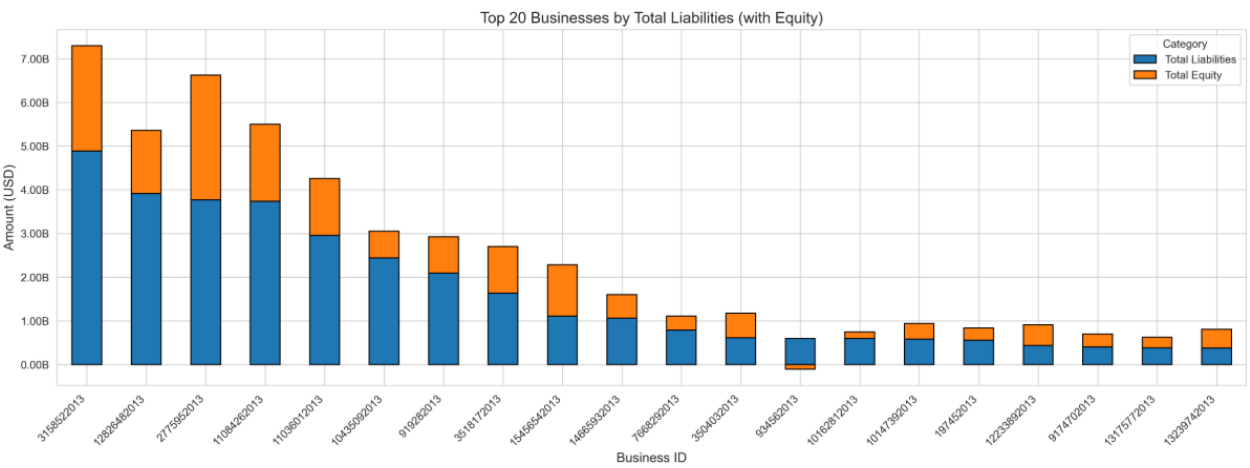
Visualization 2



Visualization 3



Visualization 4



C. Explain how customized visualizations in part B were created.

For Task 3, additional libraries were imported to create higher-quality data visualizations. The additional libraries are matplotlib.pyplot as plt, matplotlib.ticker as ticker, and seaborn as sb. The `set_style()` function was called with the 'whitegrid' option to apply a clean grid-based background and improve the readability of the visualizations.

```
5 #Import Libraries
6 import pandas as pd
7 import matplotlib.pyplot as plt
8 import matplotlib.ticker as ticker
9 import seaborn as sb
10 sb.set_style('whitegrid')
11
12 from IPython.display import display
```

For visualization 1, a horizontal bar graph was selected to help visualize the top 25 states by median total revenue. First, a new dataframe, df_viz1, was created to store the Total Revenue_median values from the df_desc_stats dataframe. These records were then sorted in descending order so the output appears organized and easily read.

Next, the bar plot is created using the .plot() method with kind = 'barh' to specify the chart type. I removed the legend, but added a descriptive title. Set the figure size to (12,8) and set the color to skyblue. Axis labels were added to identify the Business State (y) and the Median Total Revenue(x). ticker.FuncFormatter was called to apply the unit values across the x-axis.

Then, using matplotlib.pyplot, the tight_layout, savefig, and show functions are used. Tight_layout() to optimize spacing. Savefig() to save the visualization as a PNG file in the same directory as the code. Then, finally, show() to display the bar graph in the Jupyter output.

```
81 #Visualization 1: Bar graph - Top 25 States by Median Total Revenue
82 #Copy and sort median revenue data
83 df_viz1 = df_desc_stats[['Total Revenue_median']].copy()
84 df_viz1 = df_viz1.sort_values(by='Total Revenue_median', ascending=False).head(25)
85
86 #Create the bar plot
87 ax = df_viz1.plot(
88     kind='barh',
89     legend=False,
90     title='Top 25 States by Median Total Revenue',
91     figsize=(12, 8),
92     color='skyblue'
93 )
94
95 #Formatting
96 ax.set_xlabel('Median Total Revenue (USD, billions)')
97 ax.set_ylabel('Business State')
98 ax.xaxis.set_major_formatter(
99     ticker.FuncFormatter(lambda x, pos: (f"{x/1e9:.2f}".rstrip('0').rstrip('.')) + "B"))
100 ax.grid(True, axis='x', linestyle='--', linewidth=0.5, alpha=0.7)
101
102 plt.tight_layout()
103 plt.savefig("visualization1.png", dpi=300, bbox_inches='tight')
104 plt.show()
```

For Visualization 2, a scatter plot was created to visualize the relationship between profit margin and total revenue. This scatter plot was generated using Seaborn's `regplot(scatter=True)`, filtered negative Total Revenue with `data=df_input[df_input['Total Revenue'] > 0]`, and set `lowess=True` to use a smooth regression.

A new function was defined, `money_label`, to assist in formatting the x-axis for readability. The `xscale` is set to `'log'`.

Using the `matplotlib.ticker.LogLocator`, I set the location of the tick marks along the x-axis. `Ticker.FuncFormatter` applied the `money_label` function on the x-axis while `PercentFormatter` formatted the y-axis as percentages. Then, `matplotlib.pyplot` was used to add a descriptive title, axis labels, and a grid with dashed lines for readability. Similar to visualization 1, `tight_layout()`, `savefig()`, and `show()` were also used.

```

107 #Visualization 2: Scatter Plot - Profit Margin vs Total Revenue
108 ax = sb.regplot(
109     x='Total Revenue',
110     y='Profit Margin',
111     data=df_input[df_input['Total Revenue'] > 0],
112     scatter=True,
113     ci=None,
114     line_kws={'color': 'red'},
115     scatter_kws={'alpha': 0.5, 's':30},
116     lowess=True
117 )
118
119 def money_label(v, pos):
120     if v >= 1_000_000_000:
121         return f'{v/1_000_000_000:.1f}B'
122     elif v >= 1_000_000:
123         return f'{v/1_000_000:.1f}M'
124     elif v >= 1_000:
125         return f'{v/1_000:.0f}K'
126     else:
127         return f'{v:.0f}'
128
129 ax.set_xscale('log')
130 ax.xaxis.set_major_locator(ticker.LogLocator(base=10.0, numticks=12))
131 ax.xaxis.set_minor_locator(ticker.LogLocator(base=10.0, subs=(2, 5), numticks=12))
132 ax.xaxis.set_major_formatter(ticker.FuncFormatter(money_label))
133 ax.yaxis.set_major_formatter(ticker.PercentFormatter(xmax=100))
134 ax.collections[0].set_sizes([20])
135 ax.set_ylim(-25, 25)
136
137 #Formatting
138 plt.title('Do Businesses With More Revenue Have Higher Profit Margins?', fontsize=14)
139 plt.xlabel('Total Revenue (USD, log scale)', fontsize=12)
140 plt.ylabel('Profit Margin %', fontsize=12)
141 plt.grid(True, which='both', linestyle='--', linewidth=0.5, alpha=0.7)
142
143 plt.tight_layout()
144 plt.savefig("visualization2.png", dpi=300, bbox_inches='tight')
145 plt.show()

```

For visualization 3, a Histogram of Debt-to-Income Ratios across businesses was generated using Seaborn's histplot function with x set to Debt-to-Income Ratio and the data set to df_debt_income_ratio. Kde was set to true to overlay a smooth density curve on the histogram for additional insight. Similar to the previous visualizations, matplotlib.pyplot was used to add the title, labels, tight_layout(), savefig(), and show(). Matplotlib.ticker.PercentFormatter formatted the x and y axes as percentages

```
148 #Visualization 3: Histogram - Debt-to-Income Ratio Distribution
149 plt.figure(figsize=(10, 6))
150 ax = sb.histplot(
151     data=df_debt_income_ratio,
152     x='Debt-to-Income Ratio',
153     bins='fd',
154     kde=True,
155     stat='percent',
156     color='steelblue',
157     edgecolor='black',
158     kde_kws={'bw_adjust': 0.5}
159 )
160
161 ax.yaxis.set_major_formatter(ticker.PercentFormatter(xmax=100))
162 ax.xaxis.set_major_formatter(ticker.PercentFormatter(xmax=100))
163
164 plt.title('Distribution of Debt-to-Income Ratios Across Businesses', fontsize=14)
165 plt.xlabel('Debt-to-Income Ratio (%)', fontsize=12)
166 plt.ylabel('Share of businesses (%)', fontsize=12)
167
168 plt.tight_layout()
169 plt.savefig("visualization3.png", dpi=300, bbox_inches='tight')
170 plt.show()
```

For visualization 4, a stacked bar chart was created to visualize the top 20 businesses by total liabilities with equity. A new dataframe, df_viz4, was created from df_input. This dataframe was sorted in descending order by Total liabilities, and the top 20 records were selected (head(20)). Business ID is then set as the index to label each bar.

The stacked bar chart was generated using the .plot() method with kind = 'bar' and stacked = True to combine liabilities and equity for each business. Formatting included a descriptive title, labeled axes, and a legend. X-axis labels were rotated 45 degrees for readability. Finally, matplotlib.pyplot was used for labels, tight_layout(), savefig(), and show(). Matplotlib.ticker.FuncFormatter was used to apply formatting across the y-axis.

```
172 #Visualization 4: Stacked Bar Chart - Total Liabilities and Total Equity by Business
173 df_viz4 = df_input[['Business ID', 'Total Liabilities', 'Total Equity']]
174 df_viz4 = df_viz4.sort_values(by='Total Liabilities', ascending=False).head(20)
175
176 #Set index to Business ID
177 df_viz4.set_index('Business ID', inplace=True)
178
179 ax = df_viz4.plot(
180     kind='bar',
181     stacked=True,
182     figsize=(16, 6),
183     color=['#1f77b4', '#ff7f0e'],
184     edgecolor='black'
185 )
186
187 # Formatting
188 ax.set_title('Top 20 Businesses by Total Liabilities (with Equity)', fontsize=14)
189 ax.set_xlabel('Business ID', fontsize=12)
190 ax.set_ylabel('Amount (USD)', fontsize=12)
191 ax.legend(title='Category')
192 plt.xticks(rotation=45, ha='right')
193 ax.yaxis.set_major_formatter(
194     ticker.FuncFormatter(lambda x, pos: f'{x/1_000_000_000:.2f}B'))
195
196 plt.tight_layout()
197 plt.savefig("visualization4.png", dpi=300, bbox_inches='tight')
198 plt.show()
```

References

Pandas documentation#. *pandas documentation - pandas 2.3.1 documentation.* (n.d.).

<https://pandas.pydata.org/docs/index.html>

WGU Analytics Programming course materials, August 2025

Pyplot tutorial#. *Pyplot tutorial - Matplotlib 3.10.5 documentation.* (n.d.).

<https://matplotlib.org/stable/tutorials/pyplot.html>

Statistical Data Visualization#. *seaborn.* (n.d.). <https://seaborn.pydata.org/>